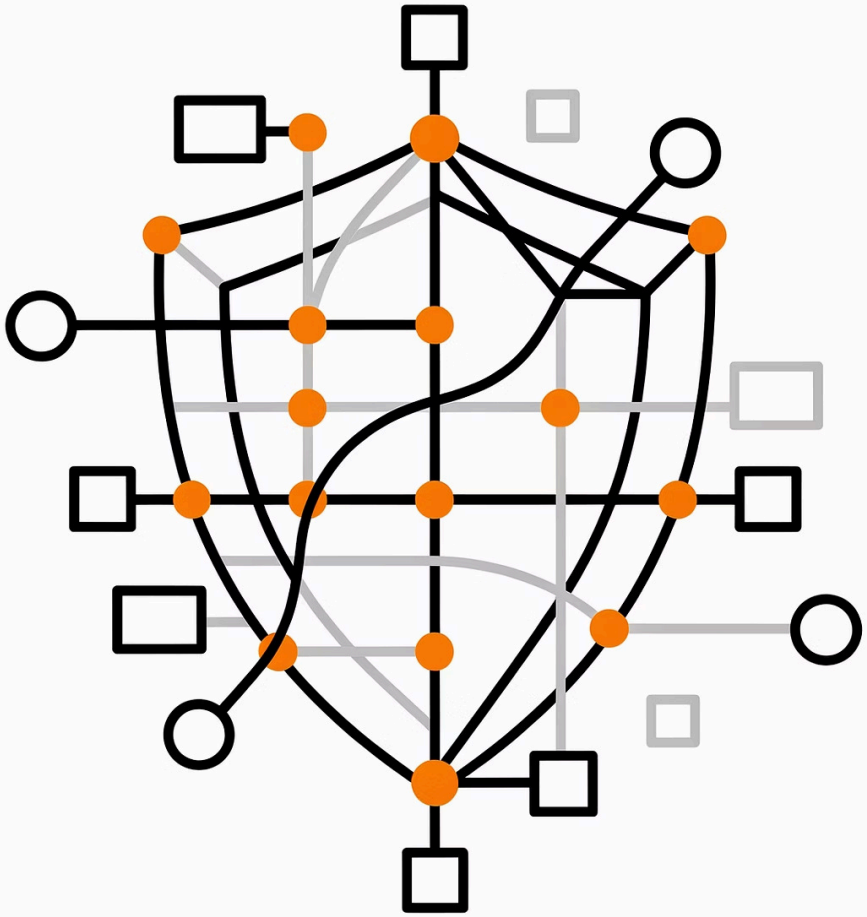


# OWASP Top 10 : A03:2021 – Injection

**Cours : Sécurité des Applications Web**

**Objectif :** Comprendre ce que sont les vulnérabilités d'injection, pourquoi elles fonctionnent toutes de la même manière, comment elles sont exploitées, et comment les prévenir.

Made with GAMMA

OWASP TOP 10 | A03:2021

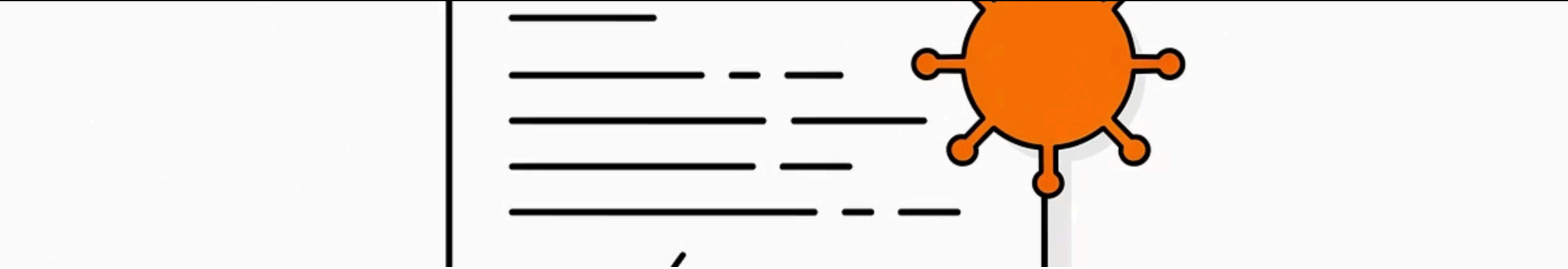
# Introduction aux Vulnérabilités d'Injection

Les vulnérabilités d'injection représentent l'une des **familles de failles les plus courantes et dangereuses** dans le développement web moderne. Classées A03 dans le Top 10 de l'OWASP 2021, elles continuent de menacer des milliers d'applications à travers le monde.

Ces vulnérabilités ne se limitent pas à un seul type de système. Elles affectent une diversité de composants critiques :

- **Bases de données** (SQL, NoSQL)
- **Systèmes d'exploitation** et shells
- **Systèmes de fichiers**
- **Moteurs de templates**
- **APIs** et services web

Made with GAMMA



# Qu'est-ce qu'une Injection ?

### Le Principe Fondamental

Une injection se produit lorsque **des données non fiables sont interprétées comme du code** par un système cible.

### Le Mécanisme

Des données contrôlées par l'utilisateur sont envoyées à un autre outil qui possède **sa propre syntaxe** et **ses caractères spéciaux**.

### L'Échec de Sécurité

L'application ne parvient pas à gérer cette transition de manière sécurisée, permettant l'exécution de code malveillant.

Cette défaillance crée une brèche où les attaquants peuvent manipuler le comportement de l'application en injectant leurs propres instructions dans des contextes où seules des données devraient exister.

Made with GAMMA

# Le Principe Universel d'Injection

**TRÈS IMPORTANT :** Toutes les vulnérabilités d'injection reposent sur la même erreur fondamentale

1

### Outil A

Une application déplace des données depuis un premier outil (source)

2

### Frontière Critique

L'Outil B possède des **caractères spéciaux différents** de l'Outil A

3

### Exploitation

Si ces caractères ne sont pas correctement gérés, l'Outil B exécute des **instructions non prévues**

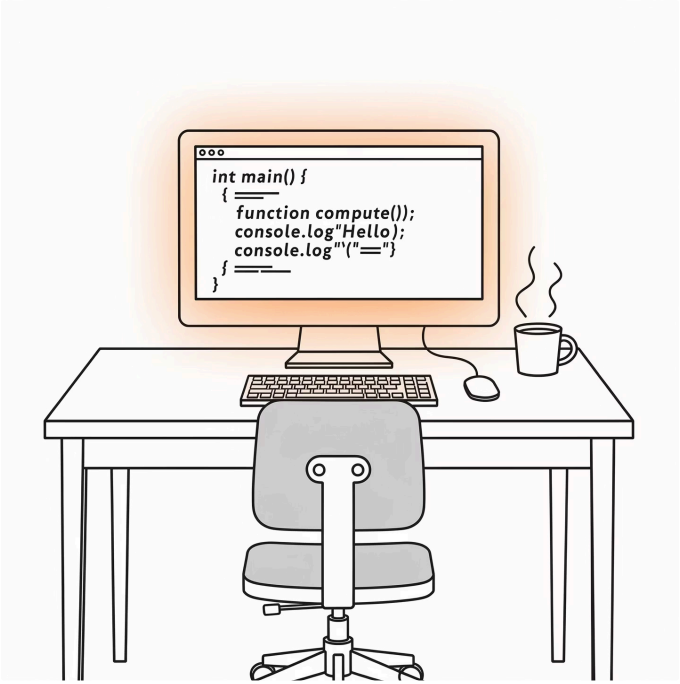
> **Injection = Échec à une frontière d'outil**

Cette compréhension unifiée est essentielle : que vous travailliez avec SQL, des shells système, ou des templates, le problème sous-jacent reste identique. Maîtriser ce principe vous permet de reconnaître et prévenir toutes les formes d'injection.

Made with GAMMA

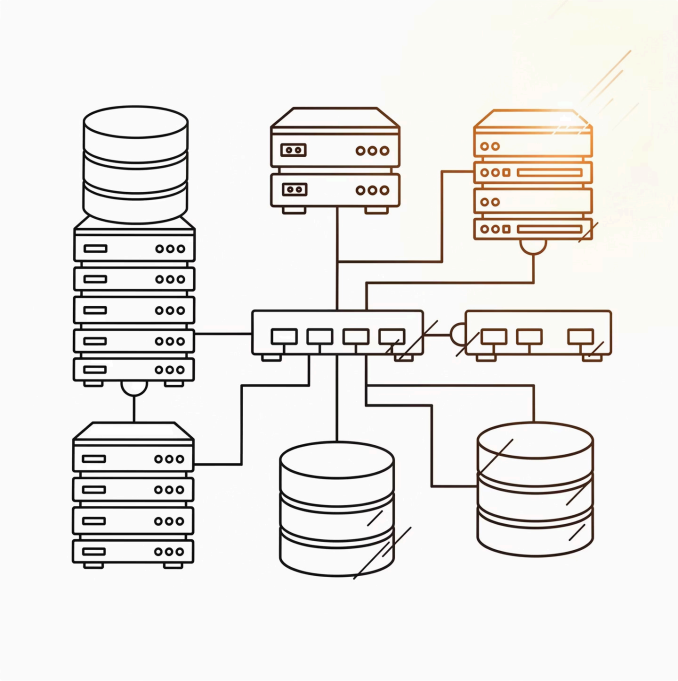
# Modèle Mental : Frontière d'Outils

## Outil A (Source)




- Langage de programmation (PHP, Java, JavaScript)
- Saisie utilisateur
- Structures de données
- Chaînes de caractères

## Outil B (Interpréteur)



- Moteur SQL
- Shell / Système d'exploitation
- Système de fichiers
- Moteur de template
- Service d'annuaire (LDAP)

---

L'injection se produit **lorsque l'Outil A envoie des données que l'Outil B interprète comme du code**. Cette frontière entre les outils représente le point de vulnérabilité critique où la distinction entre données et instructions doit être maintenue avec rigueur.

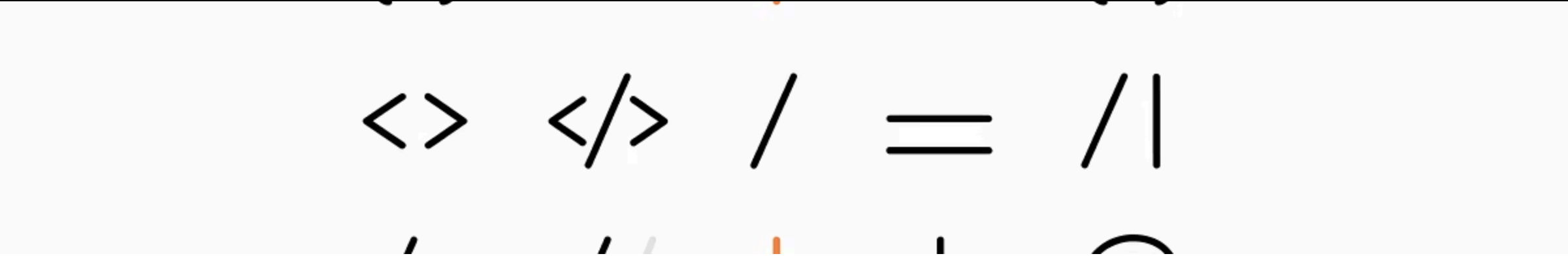
Made with GAMMA

# Types d'Injection : Même Cause Racine

Bien que les contextes diffèrent, toutes les injections partagent le même mécanisme fondamental. Voici les transitions d'outils les plus courantes où ces vulnérabilités apparaissent :

| Outil A → Outil B | Type d'Injection | Contexte |
|---|---|---|
| PHP / Java → SQL | **SQL Injection** | Requêtes de base de données |
| Code applicatif → Shell | **Command Injection** | Exécution de commandes système |
| JSON / API → Moteur NoSQL | **NoSQL Injection** | Bases de données non relationnelles |
| Chaîne → Chemin fichier | **Path Traversal** | Accès au système de fichiers |
| Données → Moteur de template | **SSTI** | Rendu de templates |
| Requête LDAP → Service d'annuaire | **LDAP Injection** | Interrogation d'annuaires |

Made with GAMMA



# Pourquoi les Caractères Spéciaux Sont Importants

## Syntaxe Propre à Chaque Outil

Chaque outil possède **sa propre syntaxe** avec des règles d'interprétation spécifiques. Ce qui est inoffensif dans l'Outil A peut devenir une instruction dans l'Outil B.

## Caractères à Risque

Des caractères apparemment anodins deviennent dangereux selon le contexte :

- `' " --` → SQL
- `; && |` → Shell
- `../` → Chemins de fichiers
- `${}` → Moteurs de templates
- `$ne, $gt` → NoSQL

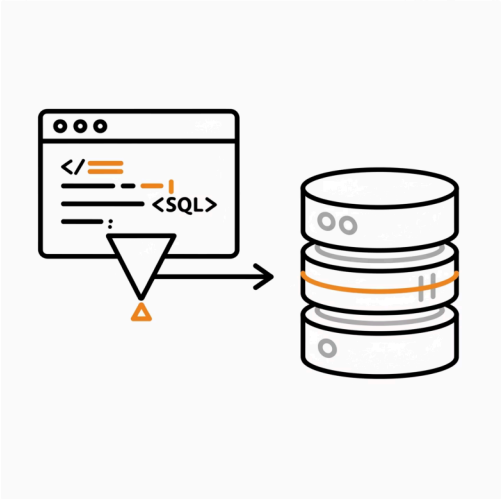
## La Conséquence Critique

**Si vous les transmettez sans modification, vous donnez le contrôle à l'Outil B.** L'attaquant peut alors manipuler la logique d'exécution et contourner les protections de l'application.

Made with GAMMA

# Scénarios d'Exemple

## SQL Injection




L'entrée utilisateur ferme une chaîne SQL, permettant à l'attaquant d'injecter de la logique SQL supplémentaire.

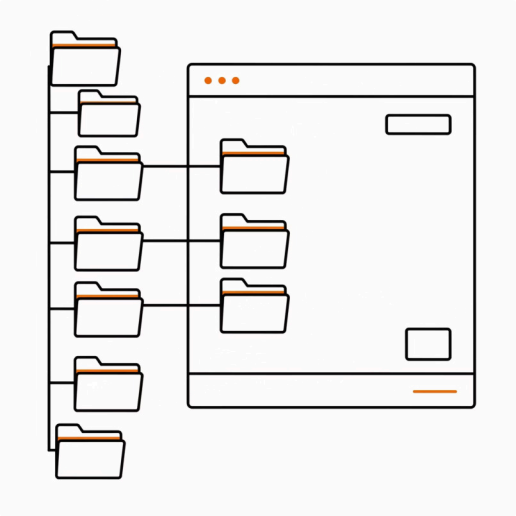
```
' OR '1'='1
```

## Command Injection

L'entrée utilisateur est ajoutée à une commande shell, l'attaquant injecte des opérateurs shell pour exécuter des commandes arbitraires.

```
; rm -rf /
```

## Path Traversal



L'entrée utilisateur est utilisée comme chemin de fichier, l'attaquant injecte `../` pour accéder à des fichiers sensibles.

```
../../etc/passwd
```

---

> Dans tous les cas : Même erreur, Outil B différent

Ces trois exemples illustrent parfaitement comment le même principe fondamental se manifeste dans différents contextes techniques. La compréhension de cette unité conceptuelle est la clé pour développer des défenses efficaces.

Made with GAMMA

# Découvrir la Vulnérabilité

## Où Chercher

Les vulnérabilités d'injection apparaissent partout où des données traversent une frontière d'outil. Voici les points d'entrée les plus courants à examiner lors d'un audit de sécurité :

### Formulaires

Champs de saisie, zones de texte, boutons radio

### Paramètres URL

Query strings, paramètres GET

### Corps JSON / XML

Données POST, APIs RESTful

### En-têtes HTTP

User-Agent, Referer, cookies

### Noms de fichiers

Uploads, téléchargements

### Filtres de recherche

Critères de tri, pagination

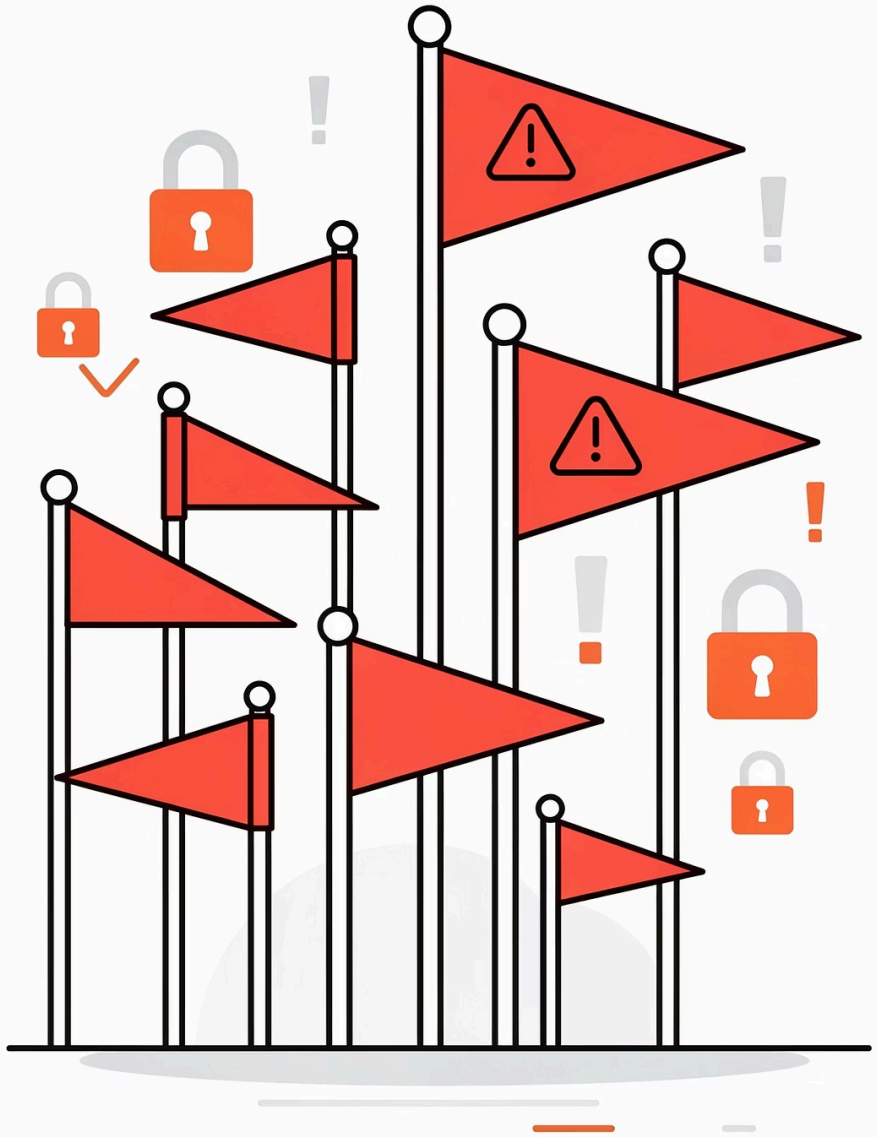
### Paramètres API

Endpoints, méthodes d'API

### Données récupérées automatiquement

Les sources de données externes, par exemple scrappées depuis le web.

Made with GAMMA



# Signes d'Alerte

### Concaténation de Chaînes

Construction dynamique de requêtes ou commandes par assemblage de chaînes de caractères

### Fonctions Dangereuses

Utilisation de fonctions à risque : `exec`, `system`, `eval`

### Messages d'Erreur Révélateurs

Erreurs exposant la syntaxe SQL, erreurs shell, chemins du système de fichiers

### Comportement incohérents

Réponses variables lors de l'injection de caractères spéciaux : `' " ; -- && || ../`

La présence d'un ou plusieurs de ces indicateurs doit déclencher une analyse approfondie du code et des tests de sécurité ciblés.

Made with GAMMA

# Comment Vérifier la Vulnérabilité

01

## Intercepter et Modifier les Requêtes

Utilisez un proxy d'interception pour capturer et manipuler les communications entre le client et le serveur.

02

## Briser la Syntaxe de l'Outil B

Injectez des caractères qui cassent la syntaxe attendue et observez les erreurs ou comportements anormaux.

03

## Injecter des Caractères Logiques

Testez des opérateurs logiques et de contrôle spécifiques au contexte cible (SQL, shell, etc.).

04

## Observer le Comportement

Analysez les réponses, temps d'exécution, et messages d'erreur pour identifier les anomalies.

05

## Examiner le Code Source

Recherchez les patterns de concaténation non sécurisée dans le code de l'application.

## Outils Recommandés

- Burp Suite / OWASP ZAP
- SQLMap
- Metasploit
- Scripts personnalisés

Made with GAMMA

# Blind Injections : Un Cas Particulier

Contrairement aux injections "classiques" où le serveur révèle des messages d'erreur ou des résultats directs, les blind injections ne fournissent aucune information visible. L'attaquant doit déduire les informations en observant des réponses subtiles du système.

## Invisibilité

Aucun message d'erreur ou de données affichées directement, forçant l'attaquant à travailler "à l'aveugle".

## Inférence par Booléens

Les informations sont extraites en envoyant des requêtes qui évaluent une condition (vraie/fausse) et en observant des changements dans la réponse HTTP (ex: page trouvée vs. page non trouvée).

## Inférence par Temps

L'attaquant injecte des commandes qui provoquent un délai si une condition est vraie. Le temps de réponse du serveur permet de confirmer la validité de la condition.

## Exploitation Complexe

Ce type d'attaque est beaucoup plus lent et technique, nécessitant souvent l'usage de scripts ou d'outils automatisés pour extraire les données caractère par caractère.

Made with GAMMA

# Outils de Détection et d'Exploitation

## Burp Suite / OWASP ZAP

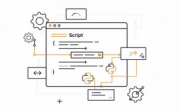
Proxies d'interception professionnels pour analyser et manipuler le trafic HTTP/HTTPS. Essentiels pour identifier les points d'injection potentiels.

## SQLMap

Outil automatisé de détection et d'exploitation des injections SQL. Supporte de nombreux SGBD et techniques d'exploitation avancées.

## Metasploit

Framework complet de test de pénétration incluant des modules pour divers types d'injection et d'exploitation post-exploitation.



## Scripts Personnalisés

Automatisation sur mesure pour tester des cas spécifiques ou des contextes applicatifs particuliers nécessitant une approche ciblée.

Made with GAMMA

# Principes d'Exploitation

Les attaquants suivent toujours une logique méthodique et reproductible. Comprendre leur processus de pensée est essentiel pour anticiper et bloquer leurs tentatives.

### Identifier une Frontière d'Outil

Localiser les points où des données transitent d'un système à un autre, particulièrement vers des interpréteurs.

### Comprendre la Syntaxe de l'Outil B

Étudier le langage et les caractères spéciaux de l'outil cible (SQL, shell, template, etc.).

### Injecter les Caractères Spéciaux

Construire des payloads exploitant les caractères de contrôle propres à l'Outil B.

### Observer le Comportement Non Prévu

Confirmer l'exploitation en vérifiant l'exécution d'instructions arbitraires ou l'accès à des données sensibles.

**Rappelez-vous, l'exploitation fonctionne parce que les données ne sont pas clairement séparées du code.** Cette confusion fondamentale est au cœur de toutes les injections réussies. À vous de réfléchir à comment exploiter ces données pour en faire du code exécutable.

Made with GAMMA

# Pourquoi la Vulnérabilité Apparaît

## Causes Racines

### Incompréhension des Frontières

Méconnaissance du concept de frontière d'outil et de ses implications sécuritaires

### Confiance Excessive dans le Filtrage

Croyance erronée que filtrer quelques caractères suffit à se protéger

### Construction Manuelle

Création artisanale de requêtes ou commandes par concaténation de chaînes

### Code Legacy ou Pressé

Applications anciennes ou développées dans l'urgence sans considération sécuritaire

### Traitement naïf des entrées utilisateurs

Supposition naïve que les utilisateurs n'enverront que des données valides

Ces facteurs se combinent souvent dans les environnements de développement réels, créant des conditions propices à l'apparition de vulnérabilités d'injection critiques.

Made with GAMMA

# Fonctionnalités à Haut Risque

Certaines parties des applications web sont particulièrement susceptibles de contenir des vulnérabilités d'injection. Une vigilance accrue est nécessaire lors de leur développement et audit.

### Systèmes d'Authentification

Les formulaires de login sont des cibles privilégiées car ils communiquent directement avec les bases de données. Une injection SQL réussie peut permettre de contourner complètement l'authentification.

### Recherche et Filtrage

Les fonctionnalités de recherche construisent dynamiquement des requêtes basées sur l'entrée utilisateur, créant de nombreuses opportunités d'injection si les paramètres ne sont pas correctement traités.

### Upload et Téléchargement de Fichiers

La manipulation de noms de fichiers et chemins d'accès peut permettre des attaques de type path traversal ou l'exécution de code via l'upload de fichiers malveillants.

### Fonctionnalités Admin/Debug

Les interfaces d'administration offrent souvent un accès privilégié aux systèmes sous-jacents et sont des cibles de grande valeur pour les attaquants.

### Toute entrée utilisateur ré-injectée

Toutes les entrées utilisateurs qui sont injectées dans un autre système (base de données, système de fichier, template, page web, etc.) sont des zones d'attaques.

Made with GAMMA

# Ce qu'il NE FAUT PAS Faire

### ❌ Listes Noires

Ne vous fiez pas aux blacklists de caractères. Les attaquants trouvent toujours des moyens de les contourner avec des encodages alternatifs.

### ❌ Suppression Manuelle

Ne tentez pas de supprimer manuellement les "mauvais caractères". Cette approche est vouée à l'échec et crée un faux sentiment de sécurité.

### ❌ Concaténation de Chaînes

N'assemblez jamais de requêtes, commandes, chemins ou templates par simple concaténation de chaînes non validées.

---

> Ces approches inefficaces sont encore trop courantes dans les applications modernes. Elles donnent l'illusion de la sécurité tout en laissant les systèmes totalement vulnérables.

La protection contre les injections nécessite des solutions structurelles, pas des rustines superficielles. Les sections suivantes présentent les méthodes éprouvées pour éliminer réellement ces vulnérabilités.

Made with GAMMA

# La Correction Universelle

SOLUTION EFFICACE

POUR TOUTES LES INJECTIONS

Il existe une approche fondamentale qui élimine les vulnérabilités d'injection, quel que soit le type d'outil cible. Cette méthode repose sur un principe simple mais puissant :

### APIs Sécurisées

Utiliser des interfaces de programmation conçues pour la sécurité

### Requêtes Paramétrées

Séparer explicitement les données des instructions SQL

### Statements Préparés

Précompiler les requêtes avec des placeholders pour les données

### Wrappers Sécurisés

Encapsuler les commandes système dans des couches de sécurité

**Le principe clé :** Permettre à l'Outil B de distinguer clairement : "Ceci est une donnée, pas du code"

Cette séparation explicite entre code et données est la seule défense véritablement efficace. Elle retire aux caractères spéciaux leur pouvoir d'altérer la logique d'exécution.

Made with GAMMA

# Exemple de Correction

## ❌ Version Non Sécurisée

```
String query = "SELECT * FROM users WHERE login = '" + login + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(query);
```

**Problème :** La variable `login` est directement concaténée dans la requête SQL. Un attaquant peut injecter : `' OR '1'='1`

## ✓ Version Sécurisée

```
String query = "SELECT * FROM users WHERE login = ?";
PreparedStatement pstmt = conn.prepareStatement(query);
pstmt.setString(1, login);
ResultSet rs = pstmt.executeQuery();
```

**Solution :** Les paramètres sont traités strictement comme des données. Les caractères spéciaux perdent leur pouvoir d'altération de la logique SQL.

---

**100%**

Protection

Efficacité des requêtes paramétrées contre l'injection SQL

**0**

Exceptions

Aucune raison valable de ne pas utiliser cette méthode

Made with GAMMA

# Challenge CTF : ParcelPulse

PRATIQUE & EXPÉRIMENTATION

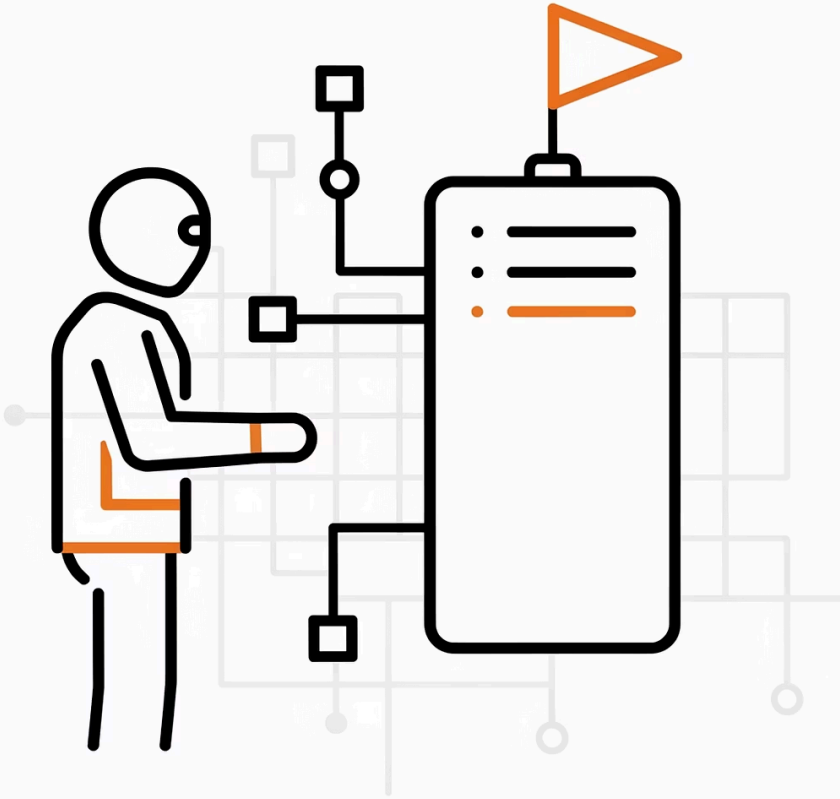
**ParcelPulse – Plateforme Logistique**

ParcelPulse gère le transport et le suivi de colis à l'échelle européenne. Mais derrière ses systèmes de suivi publics et son portail interne de support se cachent des failles… Saurez-vous exploiter le système pour découvrir les secrets de l'entreprise et récupérer les flags cachés ?

| | |
|---|---|
| 1 | **Challenge 1 : Accès au Portail Interne**<br>Obtenez des identifiants pour accéder au portail interne de support. Récupérez le premier flag et débloquez l'accès au Challenge 2. |
| 2 | **Challenge 2 : Extraction de Données Sensibles**<br>Maintenant que vous avez accès au portail interne, exploitez une faille supplémentaire pour extraire le flag final sur la machine. |

**Durée estimée :** 120 minutes

Ce challenge permet de mettre en pratique l'ensemble des connaissances acquises dans un contexte réaliste et contrôlé.



Made with GAMMA

# À Retenir Absolument

### L'injection n'est PAS un problème SQL

C'est une idée reçue dangereuse. L'injection affecte tous les types de systèmes et d'interpréteurs.

### L'injection n'est PAS spécifique à un outil

Le même mécanisme fondamental s'applique à SQL, aux shells, aux templates, aux systèmes de fichiers, et bien d'autres.

### L'injection concerne trois éléments

- Le **franchissement de frontières d'outils**
- Des **caractères spéciaux différents** entre les outils
- La **confusion entre données et instructions**

### Règle d'Or

**Si vous ne contrôlez pas l'entrée et qu'elle atteint un autre outil, vous devez supposer qu'une injection est possible.**

Cette compréhension unifiée des mécanismes d'injection vous permettra de reconnaître et prévenir ces vulnérabilités dans tous vos projets futurs, quel que soit le contexte technique.

Made with GAMMA